Architecture of C++ STL Linked List

Qiang Liu

Qishi C++ study group

PRESENTATION AGENDA

Foundations

Understanding node-based memory and pointer structures.

Operations

list methods: insertions, deletions, splices, etc.

Case Study

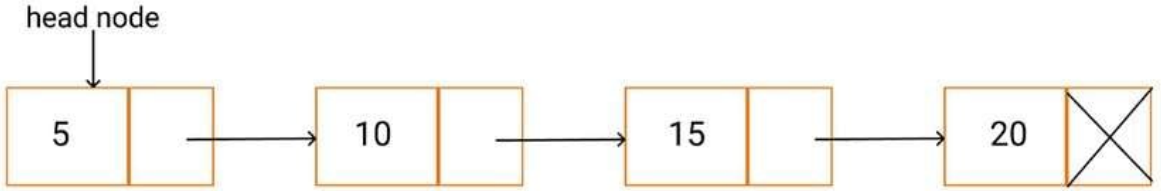
An example using list.

01. Core Concepts

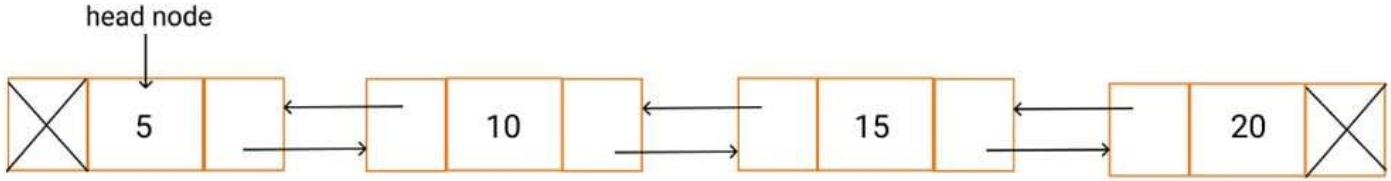
Memory Architecture & Linked Sequences

Types of Linked List

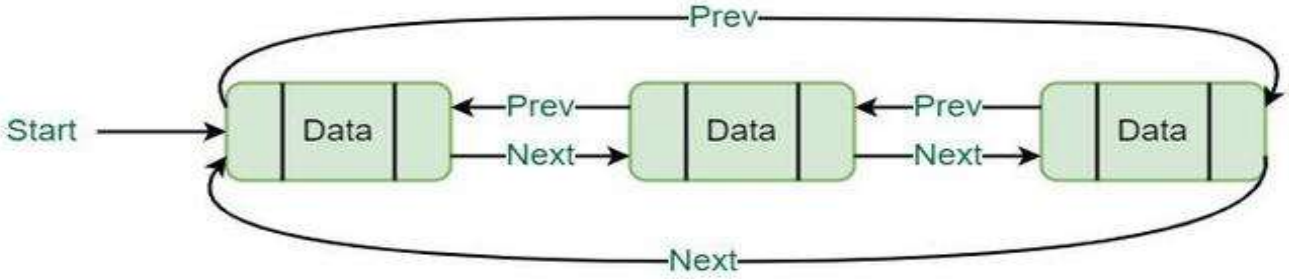
Singly Linked List



Doubly Linked List



Circular Linked List

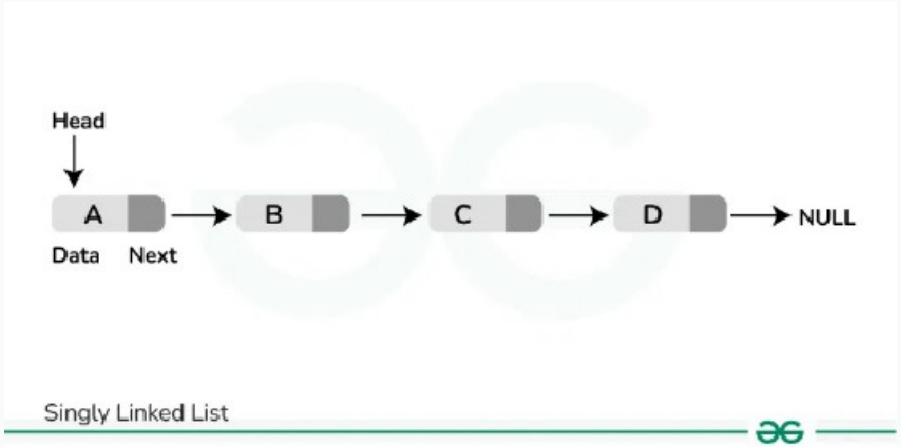


STD::FORWARD_LIST CONCEPT

The Singly-Linked Minimalist

Introduced in C++11, **std::forward_list** is designed to offer zero-overhead compared to a raw C-style linked list.

- **One Pointer:** Only stores a 'next' pointer.
- **Forward Only:** Traversal is strictly unidirectional.
- **Memory Efficient:** No backward pointer overhead.
- **No size():** Complexity is O(1), but counting is O(n).



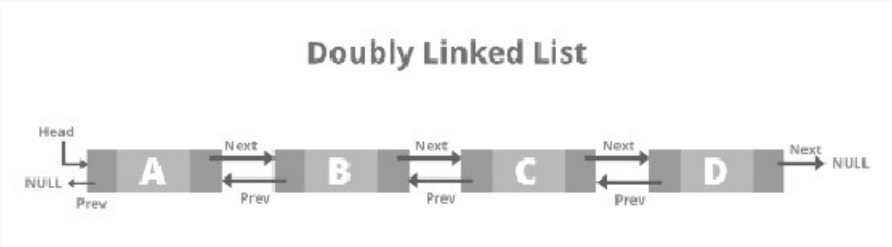
STD::LIST ARCHITECTURE

The Doubly-Linked Sequence

The standard **std::list** provides bidirectional traversal by maintaining two pointers per node.

- **Bidirectional:** Supports --iterator.
- **Stable Iterators:** Pointers remain valid after insertion/deletion.
- **Node Lifetime:** Each element is a separate heap allocation.
- **Constant Time size():** Since C++11.

```
class Node {
public:
    T data;
    Node* prev;
    Node* next;
    Node(Td) {
        data=d;
        prev=nullptr;
        next=nullptr;
    }
};
```



THE "POINTER TAX" ANALYSIS

std::list Node (64-bit)

Memory per node of T:
sizeof(T) + 8 (prev) + 8 (next)
+ heap metadata (8-16 bytes)

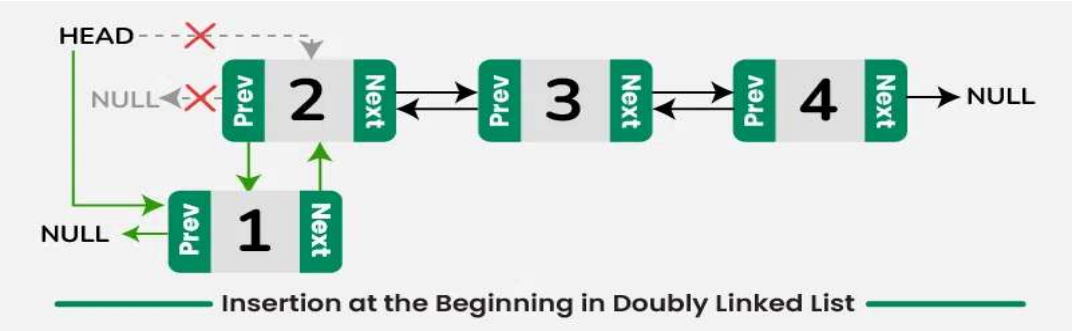
Total Overhead: ~24-32 bytes per element.

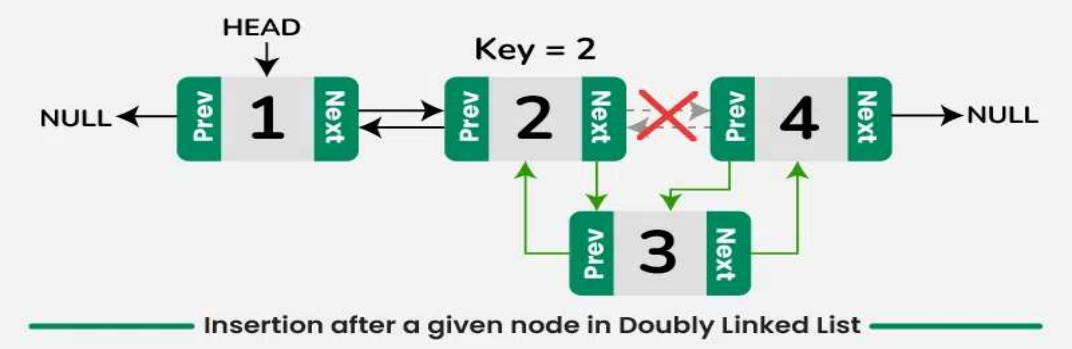
std::forward_list Node

Memory per node of T:
sizeof(T) + 8 (next)
+ heap metadata (8-16 bytes)

Total Overhead: ~16-24 bytes per element.

VISUALIZING OPERATIONS





Logical Node Insertion

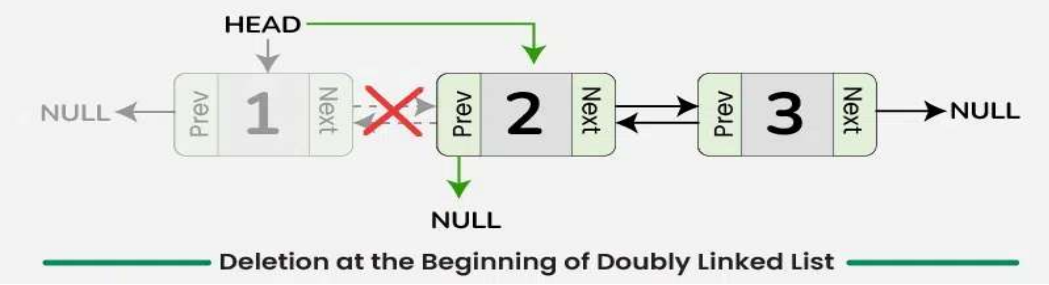
Unlike Vector, which requires shifting elements, List insertion involves simple pointer redirection.

Complexity: at any iterator position. `O(1)`

NODE DELETION

Safe & Instant Removal

Erasure in linked containers is highly efficient because it avoids memory reallocations or block copies.
When list::erase(it) is called, the container simply "stitches" the neighbors together, bypassing the target node.
Key Fact: Deletion only invalidates the iterator pointing to the erased element. All other iterators remain perfectly safe.

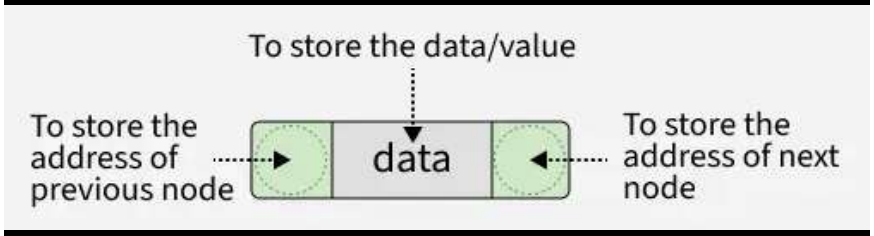


02. C++ STL std::list

Implementation and Usage

Implementation -- C++ container list

```
template <typename T>

struct Node {

    T data;
    Node* prev;
    Node* next;

    Node(const T& d) {
        data=d;
        prev=nullptr;
        next=nullptr;
    }
};
```




```
template <typename T>

class list
{
  private:
        Node* head;
        Node* tail;
        size_t sz;

  public:
    list():head(nullptr),tail(nullptr), sz(0){}

    ~list(){
        while(head)
        {
            Node* temp = head;
            head = head->next;
            delete  temp;
        }
        sz = 0;
    }

    void push_back(const T& item) {...}
    //... ...
}
```

Usage

```
#include <iostream>
#include <list>
using namespace std;

int main(){
    list<int> myList;

    myList.push_back(10);
    myList.push_back(20);
    myList.insert(myList.begin(), 5);

    cout<<" List elements: ";
    for (int n : myList) {cout<<n<<" ";}

    cout<<"\n List elements: ";
    for (auto it = myList.begin(); it != myList.end(); ++it)
        cout << *it <<" ";

    cout<<endl;
    return0;
}
```

Output:

```
List elements: 5 10 20
List elements: 5 10 20
```

C++ Container provide:

- Reusable data structures
- Consistent interfaces
- Compatibility with algorithms (sort, etc)

abstraction

You don’t worry about:

- memory management
- resizing
- internal structure

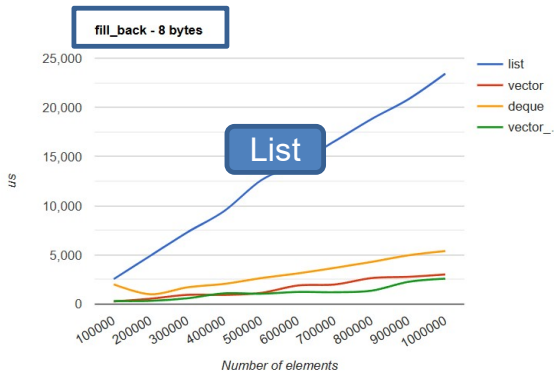
list Methods

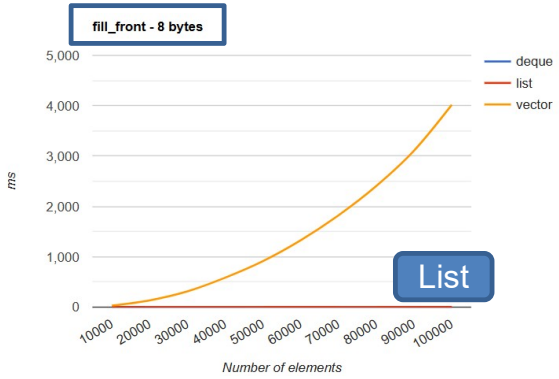
Method	Description
insert()	Inserts a new element at the iterator.
push_back()	Adds a new element at the end
push_front()	Adds a new element at the front-end
pop_back()	Deletes the last element.
pop_front()	Deletes the first element.
empty()	Checks if the list is empty.
size()	returns the number of elements
max_size()	returns the maximum size of the list.
front()	Returns the first element of the list.
back()	Returns the last element of the list.
swap()	swaps two lists when both lists are the same.

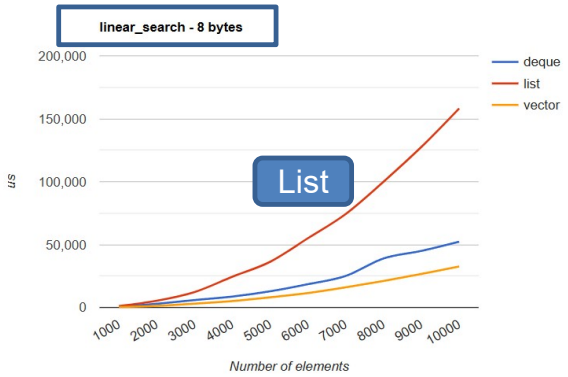
Method	Description
reverse()	Reverses the list's elements.
sort()	Sorts the elements in increasing order.
merge()	Merges two sorted lists.
splice()	Inserts a new list into the list that invokes
unique()	Removes all duplicate elements from the list.
resize()	Changes the size of the list container.
assign()	Assigns a new element to the list container.
emplace()	Inserts element at specified position.
emplace_back()	Inserts element at the end
emplace_front()	Inserts a new element at the list's beginning

COMPLEXITY COMPARISON

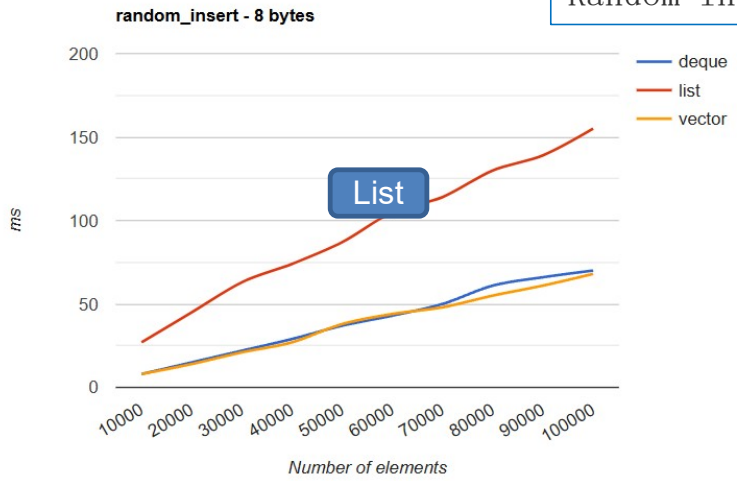
Operation	std::vector	std::list	std::deque
Access (Random)	O(1)	O(n) / O(1) with map	O(1)
Push Front	O(n)	O(1)	O(1)
Push Back	O(1) amortized	O(1)	O(1)
Middle Insert	O(n)	O(1) (with iterator)	O(n)
Search (Unsorted)	O(n)	O(n)	O(n)
Search (sorted)	O(log(n)) binary	O(n)	O(log(n))

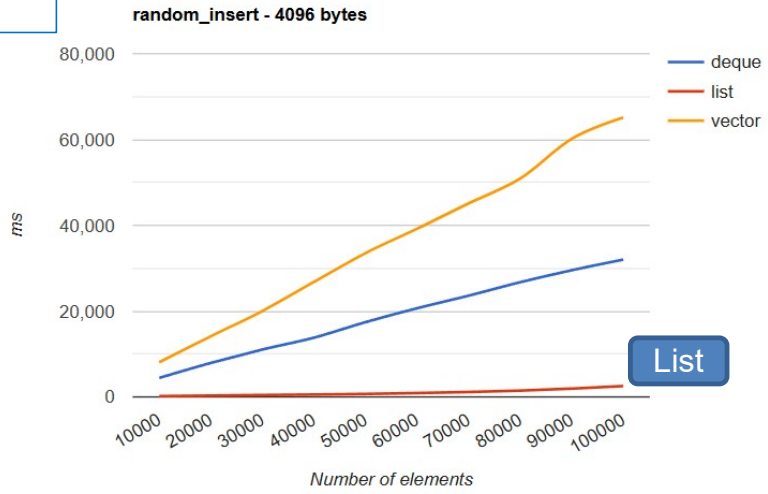






Random Insert (+Linear Search)





https://baptiste-wicht.com/posts/2012/12/cpp-benchmark-vector-list-deque.html

THE DECISION MATRIX

Choose Vector

Default choice. Best for small-to-medium datasets and random access.

Choose List

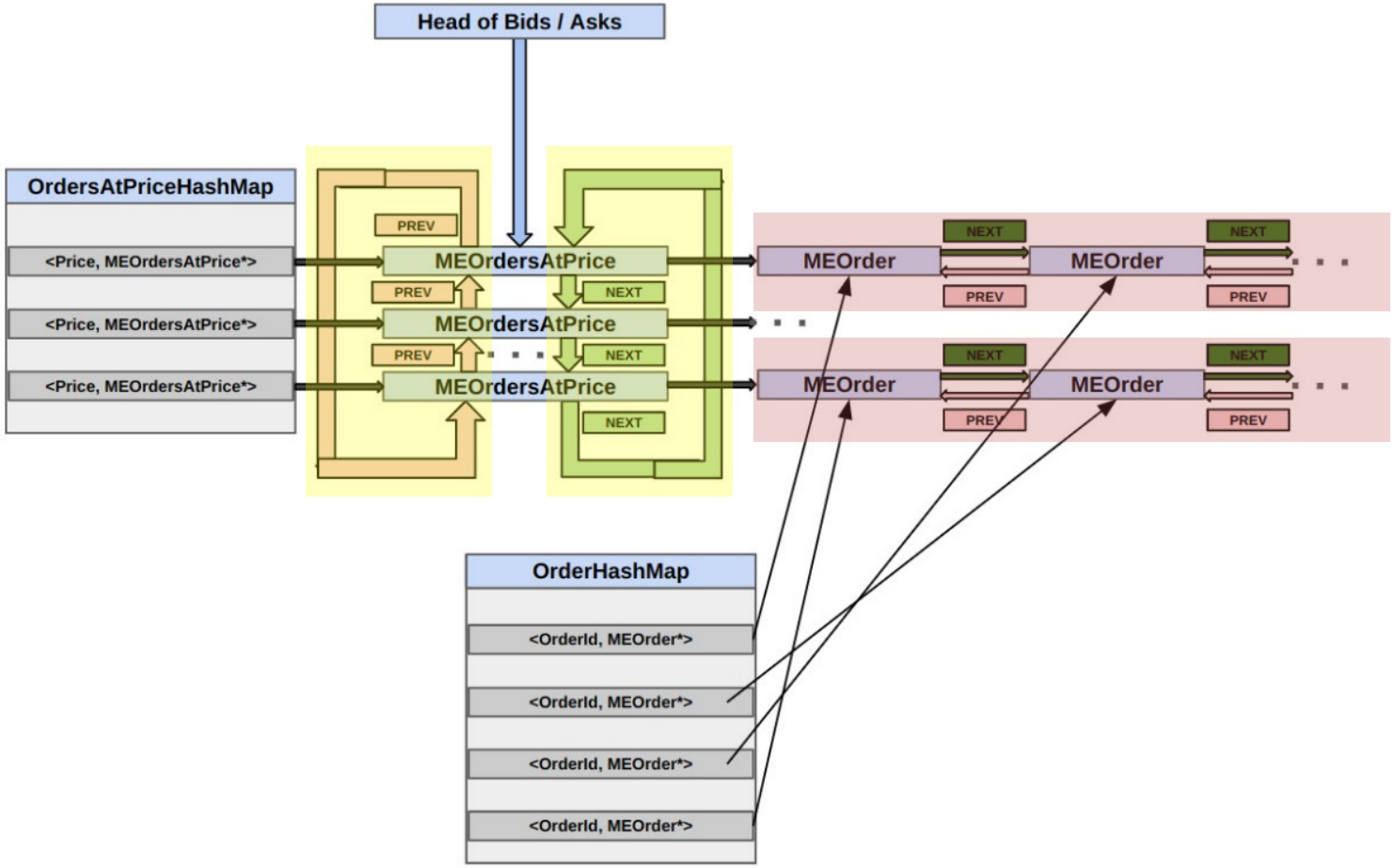
Best for frequent **middle inserts**, **large objects**, and iterator stability.

Choose Deque

Best for heavy front and back mutations (Stacks/Queues).

03. Case study

Order book


Head of Bids / Asks
OrdersAtPriceHashMap
<Price, MEOrdersAtPrice*>
<Price, MEOrdersAtPrice*>
<Price, MEOrdersAtPrice*>
PREV
MEOrdersAtPrice
NEXT
PREV
MEOrdersAtPrice
NEXT
PREV
MEOrdersAtPrice
NEXT
NEXT
MEOrder
NEXT
MEOrder
PREV
PREV
NEXT
MEOrder
NEXT
MEOrder
PREV
PREV
OrderHashMap
<OrderId, MEOrder*>
<OrderId, MEOrder*>
<OrderId, MEOrder*>
<OrderId, MEOrder*>

intrusive double linked list node (with data)

```
struct MEOrder {

    TickerId ticker_id_=TickerId_INVALID;
    ClientId client_id_=ClientId_INVALID;
    OrderId client_order_id_=OrderId_INVALID;
    OrderId market_order_id_=OrderId_INVALID;
    Side side_=Side::INVALID;
    Price price_=Price_INVALID;
    Qty qty_=Qty_INVALID;
    Priority priority_=Priority_INVALID;

    /// a node in a doubly linked list.
    MEOrder* prev_order_=nullptr;
    MEOrder* next_order_=nullptr;
  };

  struct MEOrdersAtPrice {
    Side side_ = Side::INVALID;
    Price price_ = Price_INVALID;

    MEOrder *first_me_order_ = nullptr;

    MEOrdersAtPrice *prev_entry_ = nullptr;
    MEOrdersAtPrice *next_entry_ = nullptr;
  }
```

Data Only

```
#include<list>

  struct MEOrder {

    TickerId ticker_id_=TickerId_INVALID;
    ClientId client_id_=ClientId_INVALID;
    OrderId client_order_id_=OrderId_INVALID;
    OrderId market_order_id_=OrderId_INVALID;
    Side side_=Side::INVALID;
    Price price_=Price_INVALID;
    Qty qty_=Qty_INVALID;
    Priority priority_=Priority_INVALID;

  }

  struct MEOrdersAtPrice {
    Side side_ = Side::INVALID;
    Price price_ = Price_INVALID;

    std::list<MEOrder> first_me_order_;

  }
```

Mempool Circular Double Linked List with Map

```
classMEOrderBookfinal {
  public:
    explicitMEOrderBook();

  private:
    /// Hash map from ClientId -> OrderId -> MEOrder.
    ClientOrderHashMap cid_oid_to_order_;

    MEOrdersAtPrice *bids_by_price_ = nullptr;

    MEOrdersAtPrice *asks_by_price_ = nullptr;

    /// Memory pool to manage MEOrder objects.
    MemPool<MEOrder> order_pool_;

    /// Memory pool to manage MEOrdersAtPrice objects.
    MemPool<MEOrdersAtPrice> orders_at_price_pool_;

}
```

C++ STL list

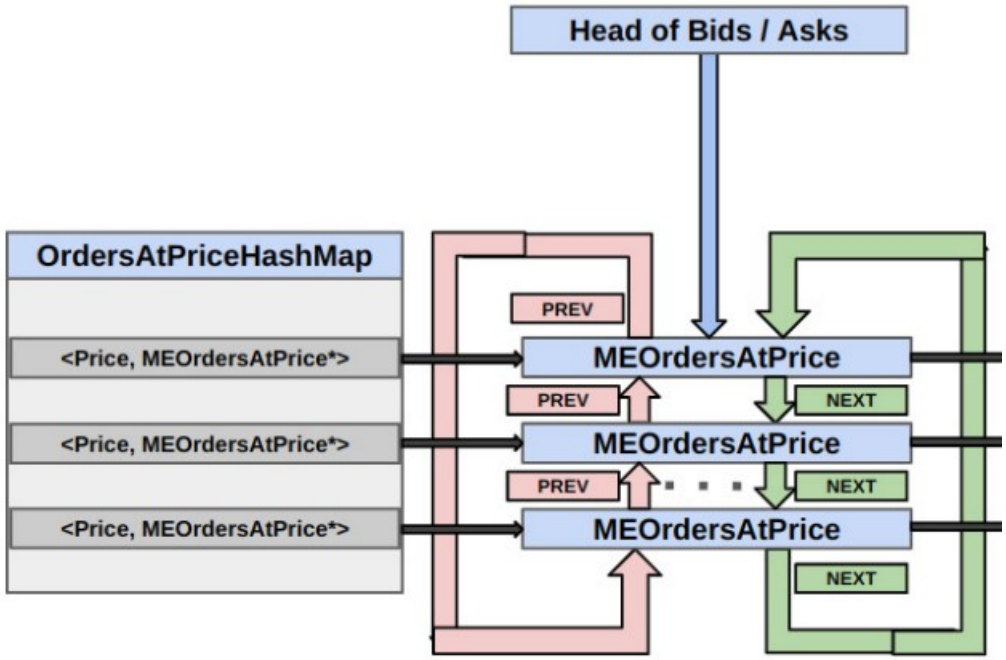
```
classMEOrderBookfinal {
  public:
    explicitMEOrderBook();

  private:
    /// Hash map from ClientId -> OrderId -> MEOrder.
    ClientOrderHashMap cid_oid_to_order_;

    std::list<MEOrdersAtPrice> bids_by_price_;

    std::list<MEOrdersAtPrice> asks_by_price_;


 }
```

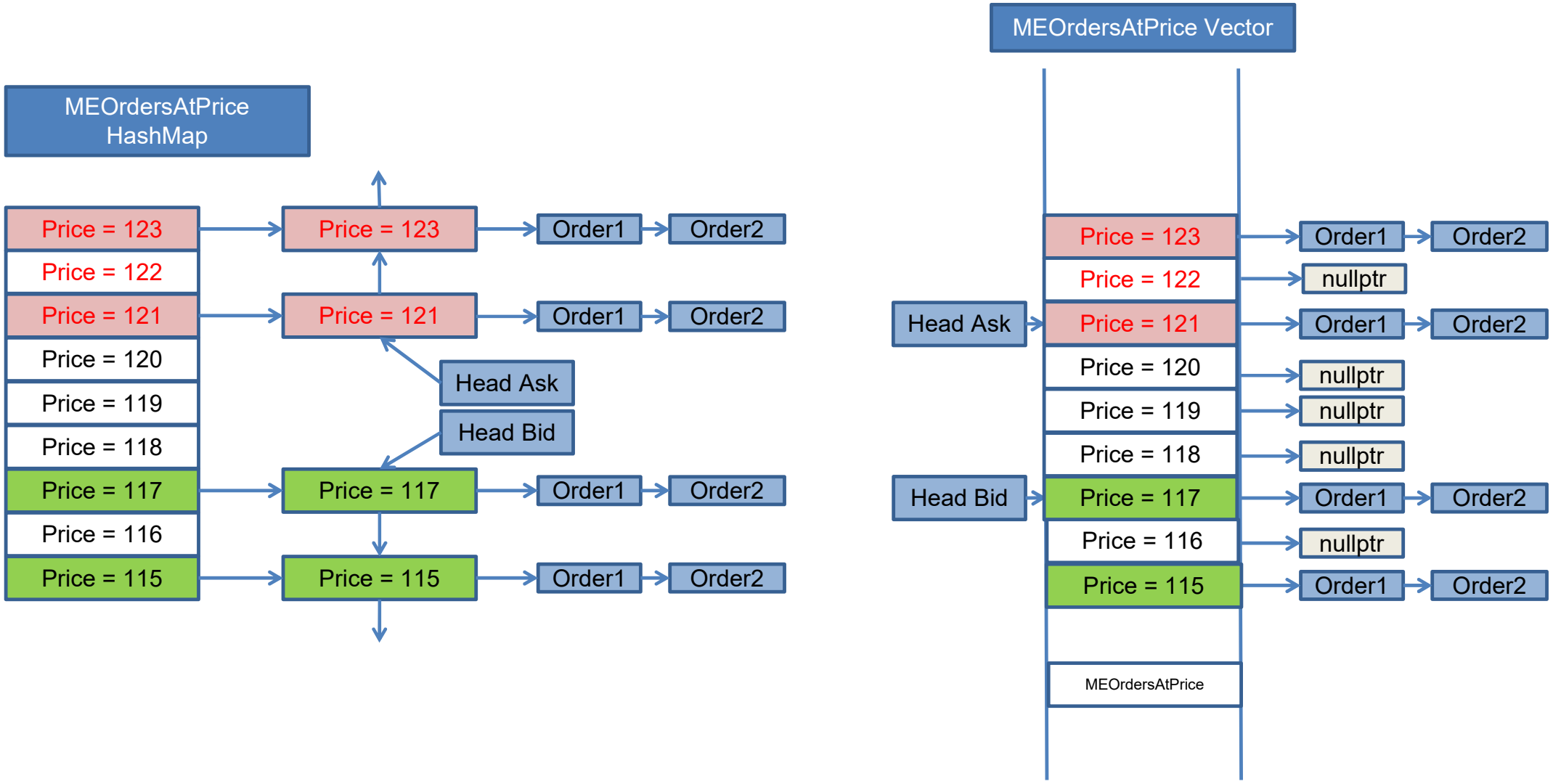
Mempool Circular Double Linked List with Map

```
autoMEOrderBook::add(ClientId client_id ...) noexcept->void {
    auto order=order_pool_.allocate(ticker_id, ...);
    addOrder(order);
  }

auto addOrder(MEOrder* order) noexcept {
  auto orders_at_price=getOrdersAtPrice(order->price_);

  if (!orders_at_price) {
    // new single node circular list
    order->next_order_=order->prev_order_=order;

    // new order_at_price node
    auto new_orders_at_price =
          orders_at_price_pool_.allocate(...);

    addOrdersAtPrice(new_orders_at_price);

  } else {
    auto first_order=orders_at_price->first_me_order_;
    // push_back
    first_order->prev_order_->next_order_=order;
    order->prev_order_=first_order->prev_order_;
    order->next_order_=first_order;
    first_order->prev_order_=order;
  }

  cid_oid_to_order_.at(order->client_id_)
       .at(order->client_order_id_) =order;
}
```

C++ STL list with Map

```
auto MEOrderBook::add(ClientId client_id ...) ->void {




    auto orders_at_price=getOrdersAtPrice(order->price_);

    if (!orders_at_price) {

        std::list<MEOrder> first_order{new MEOrder(...)};


        auto new_orders_at_price =new MEOrderAtPrice(...);

        addOrdersAtPrice(new_orders_at_price);

        cid_oid_to_order_.at(order->client_id_)
         .at(order->client_order_id_) =first_order.end();

    } else {
        auto first_order=orders_at_price->first_me_order_;
        first_order.emplace_back(ticker_id, ...);
    }

        cid_oid_to_order_.at(order->client_id_)
         .at(order->client_order_id_) =first_order.end();

}
```


Head of Bids / Asks
OrdersAtPriceHashMap
<Price, MEOrdersAtPrice*>
<Price, MEOrdersAtPrice*>
<Price, MEOrdersAtPrice*>
PREV
MEOrdersAtPrice
PREV
NEXT
MEOrdersAtPrice
PREV
NEXT
MEOrdersAtPrice
NEXT


MEOrdersAtPrice
HashMap
Price = 123
Price = 122
Price = 121
Price = 120
Price = 119
Price = 118
Price = 117
Price = 116
Price = 115
Price = 123
Price = 121
Price = 117
Price = 115
Order1
Order2
Head Ask
Head Bid
MEOrdersAtPrice Vector
Price = 123
Price = 122
Price = 121
Price = 120
Price = 119
Price = 118
Price = 117
Price = 116
Price = 115
Order1
Order2
nullptr
Head Ask
Head Bid
MEOrdersAtPrice

MODERN C++ UPDATES

- **C++20 Ranges:** Lists now support Range-based construction and assignment natively.

```
std::vector<int> v = {1, 2, 3, 4};
std::list<int> lst(v.begin()+1, v.end());
```

- **C++23 Prepend Range:** New prepend_range() and append_range() member functions for bulk node movement.

```
std::list<int>lst= {1, 2, 3};
std::vector<int> v = {4, 5, 6};
lst.insert(lst.end(), v.begin(), v.end());
```

- **C++26 Constexpr:** std::list is becoming increasingly available in constexpr contexts, allowing compile-time sequences.
- **Allocator Awareness:** Better support for PMR (Polymorphic Memory Resources) to mitigate allocation fragmentation.

```
std::pmr::monotonic_buffer_resource pool;
std::pmr::list<int> lst(&pool);
```

1. monotonic_buffer_resource
2. unsynchronized_pool_resource
3. synchronized_pool_resource
4. new_delete_resource()

SUMMARY CHECKLIST

- Linked containers are **node-based**, not contiguous.
- **forward_list** is the space-optimized singly-linked option.
- **list** is the bidirectional option with stable iterators.
- Performance is limited by **cache misses** and **allocation overhead**.
- Complexity is **O(1)** for **insert/delete** but **O(n)** for traversal.

Questions & Answers

Technical Reference: ISO/IEC C++ Standard Sequence Containers

Thank you for your attention.